

Unit of Work: RedRock Email Classifier

Problem statement (in my own words)

RedRock, a financial services company, needs incoming client emails routed to the right internal department automatically. Misrouting matters here in a way it would not for a generic inbox: this is a regulated industry, and a claim routed to the wrong team, or a compliance-sensitive email that never reaches anyone, can cause real delay and regulatory exposure. I was given 44 labeled sample emails (`train/` plus `train_labels.csv`) and 12 unlabeled ones (`test/`) and asked to build a classifier over five categories (Account Management, Investment Advisory, Loan Processing, Insurance Claims, Other) that outputs both a predicted category and a confidence score, in a way that runs end to end from raw files to a results CSV using only open source tooling.

Lab book

Data first

Before picking a model I looked at what I actually had: 44 labeled emails across 5 classes (13, 12, 7, 6, 6; mildly imbalanced but not pathological), and 12 unlabeled test emails. Each `.html` file is a rendered "email client" wrapper around a `data-field`-tagged metadata block and a *nested*, independently parseable HTML fragment holding the actual message. `src/ingestion.py` pulls subject, sender and date out of the `data-field` markers and the message text out of the nested `<body>`, rather than treating the whole file as flat text. Treating it as flat text would let the CSS, a duplicated `<title>`, and layout markup pollute the input.

Assumption flagged: the on-disk filename (`email_10.html`) is *not* a stable identifier: train and test each restart numbering at 1, so `train/email_1.html` and `test/email_1.html` would collide if I used the filename as `email_id`. Each file also carries an internal `data-field="email_id"` value, and I verified it is unique across all 56 files combined (1 through 56, no collisions). I used that internal field as the `email_id` column in the output, not the filename, since it is the only one of the two that is actually a valid identifier. This is exactly the kind of spec ambiguity worth documenting rather than guessing at silently.

Choosing the architecture

The original plan going into this was an LLM API as the primary classifier (zero or few shot, with the model's own confidence or an ensemble-agreement score), with a classical TF-IDF baseline for comparison. Two things in the actual environment changed that.

1. No API key was available in the environment this has to run in, and the spec is explicit that the solution "must be written using only open source tools... so we can run your solution and check your results." A hosted LLM call is a hard external dependency a reviewer cannot satisfy without their own paid key; that is a real risk to whether this even runs for the grader, not a stylistic preference.
2. 44 labeled examples is not enough to fine-tune anything. Whatever the primary approach was, it needed to work well with roughly 9 examples per class.

So the primary approach became a frozen sentence-embedding encoder (`all-MiniLM-L6-v2`) with a nearest-centroid classifier: embed each labeled email, average the embeddings per class into a centroid, and classify a new email by cosine similarity to each centroid (softmax over similarities gives the confidence score). This needs no training beyond averaging a handful of vectors, captures *semantic* similarity that a bag-of-words model cannot (paraphrase and synonymy: for example, "transfer my account" and "move my funds to another institution" should land near each other), and runs fully offline after a single roughly 90MB model download. It is the closest offline substitute for "LLM-based understanding" that does not require an API key, and it directly answers the spec's explicit allowance for "LLMs or other NLP techniques."

TF-IDF plus Logistic Regression stayed in as the baseline it was always meant to be: the naive benchmark the smarter approach has to beat, and a useful sanity check on whether the embedding classifier earns its extra complexity.

What I would have built if an LLM API were viable

Worth being explicit about the road not taken: with an API key available, a few-shot LLM classifier (one example per category in the prompt, structured JSON output for category, reasoning and confidence) would likely have handled ambiguous or boundary-case emails better than either approach here, since it can reason about content rather than just measuring similarity to a handful of labeled examples. I would treat it as a natural phase two (see Future Work).

Confidence scores

- Baseline (TF-IDF plus Logistic Regression): `predict_proba` output for the winning class.
- Embedding classifier: softmax over cosine similarities to each class centroid (temperature 0.05, chosen empirically so the softmax is peaked enough to be informative without being close to one-hot).

Both are bounded in $[0, 1]$ by construction (tested explicitly; see `tests/test_classifier_baseline.py::test_confidence_score_within_bounds` and the embedding equivalent). Both classifiers also expose a `predict_proba` method returning the full probability vector over all five classes, which is what makes the calibration work in the next section possible.

Evaluation methodology

44 labeled examples is too few for a single held-out validation split to be trustworthy: a 20 percent split leaves only one or two examples per class in validation. A single pass of cross-validation is better, but its headline numbers still depend somewhat on how that one random fold assignment happened to fall. The evaluation therefore uses 5-fold stratified cross-validation repeated 10 times, with each labeled email's out-of-fold probability vector averaged across all ten runs

(`src/evaluate.py::_pooled_out_of_fold`), and reports accuracy, macro-F1, per-class precision and recall, a bootstrap confidence interval on accuracy, and calibration metrics before and after temperature scaling. See Statistical Rigor below for the calibration, risk-coverage, conformal-prediction feasibility and cost-weighted error additions built on top of this.

Results

All figures below come from the repeated, pooled cross-validation described above, on the 44 labeled train emails. The 12 emails in `test/` are unlabeled; they are the emails to be classified, not a scoreable test set, so no honest metric can be computed on them.

Model	Accuracy	95% bootstrap CI	Macro-F1	ECE, raw to calibrated
TF-IDF + Logistic Regression (baseline)	0.932	0.841 to 1.000	0.904	0.496 to 0.069
Sentence-embedding + nearest-centroid (primary)	0.955	0.886 to 1.000	0.945	0.105 to 0.034

The embedding classifier wins on accuracy, macro-F1, and calibration. Its cross-validation is now also inexpensive: embeddings are computed once and cached rather than recomputed on every fold (see Statistical Rigor), so a full run of `evaluate.py`, including one-time model loading and encoding of every email, completes in under a minute end to end.

Per-class breakdown. Both models are near-perfect on Insurance Claims, Investment Advisory and Loan Processing (F1 0.92 to 1.00), which have distinct, consistent vocabulary ("claim", "premium" versus "portfolio", "fund" versus "loan", "repayment"). Both are noticeably weaker on Other:

Model	Other precision	Other recall	Other F1
Baseline	1.00	0.50	0.67
Embedding	1.00	0.67	0.80

This is expected: Other is a catch-all with no coherent internal theme, so both a bag-of-words model and a semantic-similarity model struggle to build a meaningful class centroid or set of discriminating tokens for it. The embedding classifier's better recall here is a real finding, not noise: it is genuinely capturing semantic distance from the other four, more coherent classes, which a token-overlap model cannot.

Statistical rigor

Four additions strengthen this evaluation beyond a single point estimate. Two of them (the character n-gram feature and the calibration and feasibility methodology below) were adopted after reviewing an alternative approach to this same challenge; see Comparative Review of an Alternative Approach for exactly what was reused and what was not.

Bootstrap confidence intervals. With only 44 labeled examples, a point accuracy estimate implies more precision than the sample supports. A 5,000-resample bootstrap on the out-of-fold correctness array gives the primary embedding classifier a 95 percent confidence interval of 0.886 to 1.000 around its 0.955 point accuracy, and the baseline an interval of 0.841 to 1.000 around 0.932. Reporting the interval, not just the point estimate, is part of being honest about what this evaluation can and cannot claim (`calibration.bootstrap_accuracy_ci`).

Calibration, verified against leakage. Both classifiers' raw confidence scores are miscalibrated in different directions. The baseline is badly under-confident (expected calibration error, ECE, of 0.496 before scaling), while the embedding classifier starts

much closer to calibrated already (ECE 0.105). A single-parameter temperature fit on the pooled out-of-fold probabilities (`calibration.fit_temperature`) brings both down substantially: the baseline to ECE 0.069, the embedding classifier to ECE 0.034. Because fitting the temperature on the same probabilities used to measure it risks overstating the gain, a nested check (`evaluate.py::nested_temperature_check`) fits the temperature only on an inner cross-validation of each outer training fold and measures ECE on the untouched outer fold, repeated across 25 outer evaluations. The baseline's calibration gain holds up fully out of sample: ECE falls from 0.505 to 0.091, an improvement in 100 percent of the 25 outer evaluations. The embedding classifier's smaller raw miscalibration leaves less room to improve, and the nested check shows a correspondingly smaller but still real gain, from 0.087 to 0.074, improving in 80 percent of evaluations. The calibrated confidence score, not the raw softmax or `predict_proba` output, is what is written into `results/predictions.csv` and the per-model prediction files.

Risk-coverage and an evidence-based operating threshold.

`calibration.risk_coverage_table` sweeps the auto-routing confidence threshold and reports coverage, accuracy on the auto-routed emails, and the resulting misroute count at each level. For the primary embedding classifier, a threshold of 0.90 auto-routes 86 percent of emails (38 of 44) with zero misroutes observed in cross-validation; the baseline needs a threshold of 0.95 to reach zero misroutes, at 70 percent coverage. This table, not intuition, is what an operating threshold should be chosen from. Full tables for both models are in `results/evaluation_report.txt` .

Conformal prediction: currently infeasible, and by how much. A distribution-free, class-conditional coverage guarantee (Mondrian conformal prediction) would be a stronger, more regulator-defensible way to bound a routing decision than a heuristic confidence threshold. It is not available yet on this data: split conformal prediction requires at least $\text{ceil}((1 - \alpha) * (n + 1)) \leq n$ calibration examples per class, which works out to 9 per class for 90 percent coverage and 19 per class for 95 percent (`calibration.conformal_min_n`). The smallest class in this training set, Other, has 6 labeled examples. That is a concrete, quantified case for collecting more labels in the smallest classes, rather than a vague one, and a specific number to plan around.

Cost-weighted error. Treating every misclassification as equally bad understates the cost of misrouting a regulated category. Weighting each error by an illustrative per-class cost (Insurance Claims 5, Loan Processing 4, Investment Advisory 3, Account Management 2, Other 1) gives 0.045 per email for the embedding classifier and 0.068 per email for the baseline (`calibration.cost_weighted_error`). These weights are illustrative and should be replaced with real ones agreed with compliance before use.

Comparative review of an alternative approach

A colleague independently attempted this same challenge and produced a solution built with a different AI assistant end to end (`redrock_email_classification/` in the shared workspace, not part of this project). Reviewing that work was useful, and this section states plainly what was reused from it, what was tested and rejected, and where this submission's own numbers come from.

Their approach: TF-IDF features (word 1 to 2 grams and character 2 to 5 grams) into a multinomial logistic regression, with an explicit temperature-scaled confidence score, evaluated with repeated stratified cross-validation, and reported alongside bootstrap confidence intervals, a risk-coverage threshold sweep, and a conformal-prediction feasibility calculation. On the same 44 labeled emails, they reported an accuracy of 0.932 and a macro-F1 of 0.904. Those are their own reported figures from their own code, not reproduced or verified here.

What this project adopted, reimplemented independently in `src/calibration.py` and `src/evaluate.py` , and applied to both of this project's own classifiers:

- Temperature-scaled calibration, extended here with the nested, leakage-safety check described above.
- Bootstrap confidence intervals on accuracy.
- The risk-coverage table for choosing an auto-routing threshold.
- The conformal-prediction feasibility calculation.
- The cost-weighted error metric.
- The character n-gram feature for the TF-IDF baseline. This was not taken on faith: an ablation on this project's own cross-validation confirmed a genuine macro-F1 gain (0.864 without character n-grams, 0.904 with them), and only then was it kept as the default (see Assumptions and Trade-offs).

What was deliberately not adopted: a reject-option framing, training a model only on the four business categories and assigning Other whenever confidence falls below a threshold. Both explorations tested this independently and rejected it for the same reason: Other, in this dataset, is not an open-set residual class. It has its own coherent vocabulary as a set of RedRock broadcast emails (HR notices, security alerts, satisfaction surveys), so a classifier can learn it directly, and raising a rejection threshold only pushes real business mail into the review queue instead of catching genuinely novel input. Two independent implementations, using different base models and different tooling, reaching the same

diagnosis is itself useful evidence that this is a property of the dataset rather than an artifact of a particular model.

Every number reported for this submission's own results, including all of the results in this document, comes from this project's own code in `Email_Classifier/src` and its own cross-validation runs. The colleague's project is cited here for comparison and attribution only.

Outcomes and results

The embedding-based classifier (`results/predictions.csv` , `results/predictions_embedding_centroid.csv`) is the primary submission: 5-fold, 10-repeat cross-validation gives an accuracy of 0.955 (95 percent CI 0.886 to 1.000) and a macro-F1 of 0.945, with confidence scores that are temperature-calibrated and verified against leakage (see Statistical Rigor). The TF-IDF baseline (`results/predictions_baseline_tfidf_logreg.csv`) is included for comparison and scored 0.932 accuracy and 0.904 macro-F1, a reasonable model in its own right, but one whose raw confidence needed far more correction to become a trustworthy routing signal. Both classifiers' weakest class is Other, for the structural reason described above, not a bug.

How I would measure success in production, and what would improve accuracy

Measuring success, given the regulatory stakes of misrouting:

- Track precision and recall *per class*, not just overall accuracy. A false positive that routes a claim to Other is a worse failure than confusing two adjacent advisory categories, so the cost of an error should weight the metric, not just its existence (see the cost-weighted error metric above, which formalizes exactly this).
- Track calibration over time (expected calibration error and a reliability diagram), not just accuracy. A routing system should be able to say "I am not sure, send this to a human" rather than confidently misfiling something. A model that is honest about its uncertainty is safer to automate around than one that is occasionally very wrong with high confidence. Refit and re-verify the temperature (with the nested check demonstrated above) whenever the model is retrained.
- Maintain a human-in-the-loop review queue for low-confidence predictions, informed directly by the risk-coverage table above, and audit a sample of *high-*

confidence predictions periodically to catch silent drift. Confidently wrong is the failure mode that costs the most in a regulated environment.

- Log every prediction with its input, category, confidence, and, if reviewed, the human-corrected label, both for retraining and because financial email handling is subject to recordkeeping obligations. SEC Rule 17a-4 requires broker-dealers to retain business communications, with immediate accessibility for two years and six-year total retention, and 2026 FINRA guidance extends this scrutiny explicitly to AI-assisted tooling (see sources below). An automated classifier making routing decisions on regulated correspondence should itself produce an auditable decision trail, not just a label.

What would improve accuracy and compliance value most, with more data:

- More labeled examples per class. 44 total is enough to demonstrate the approach, not to fully validate it; the bootstrap confidence interval on accuracy is wide, and the smallest class (6 examples) is well short of the 9 to 19 per class a formal conformal-prediction guarantee would need (see Statistical Rigor).
- Sender domain and historical sender-to-category patterns. A repeat sender previously routed to Loan Processing is informative prior information the current model discards entirely.
- Attachment presence and type, and structured metadata already present in some financial email systems (claim numbers, account numbers, policy references), as explicit features rather than relying on the model to infer them from free text.
- Thread and reply-chain context. A reply in an existing thread should inherit strong priors from that thread's category.
- An explicit escalate-to-human category or confidence floor, since in this domain a wrong high-confidence prediction is worse than an honest "uncertain" (this is already implemented as the risk-coverage operating threshold above; the point here is to keep tightening it as more data arrives).

Sources: [FINRA Email Retention Requirements \(2026 Guide\)](#), [Smarsh](#), [Understanding Model Calibration](#), [arXiv:2501.19047](#)

Assumptions and trade-offs (consolidated)

1. Used the internal `data-field="email_id"` as the output identifier, not the on-disk filename, because filenames collide between `train/` and `test/` while the internal field is globally unique. See Lab Book.

2. Substituted a local sentence-embedding classifier for the originally planned LLM-API-based classifier, because no API key was available in the run environment and the spec requires the solution to run end to end on open source tooling alone. Documented above under Choosing the Architecture.
3. Used 5-fold stratified cross-validation, repeated 10 times and pooled per sample, rather than a single train and validation split or a single cross-validation pass, because 44 examples across 5 classes makes a single split, or a single fold assignment, noisy.
4. Subject line is included twice in the text fed to both classifiers (`Email.text` in `ingestion.py`) to weight it slightly higher than the body for the sparse TF-IDF representation; this is a no-op for the dense embedding model but kept consistent across both for a fair comparison.
5. Added character n-gram TF-IDF features to the baseline after reviewing a colleague's independent implementation that used them, kept only after an ablation on this project's own cross-validation confirmed a genuine macro-F1 gain (0.864 to 0.904). See Comparative Review above.
6. Confidence scores are calibrated with a single-parameter temperature fit on out-of-fold probabilities, validated with a nested holdout to rule out leakage. The calibrated score, not the raw model output, is what is written to `results/predictions.csv` .
7. Sentence embeddings are computed once and cached by `email_id` , rather than recomputed on every cross-validation fold, since the evaluation now refits the embedding classifier on the order of several hundred times (repeated cross-validation plus the nested calibration check), and re-encoding the same text with the transformer that many times would be impractical without it.
8. Tested classification using only sender and date received, no subject and no body, as a check against published findings that email header features can rival full-content classification. On this dataset it scored 0.159 accuracy, worse than always predicting the majority class (0.295), and was not adopted. A handful of RedRock domain addresses do correlate with a department (for example `compliance@redrock.com` with Investment Advisory), but most senders are unrelated personal addresses, and 44 examples is not enough for the model to separate the two reliably. This is a property of this synthetic sample, not a reason to expect the same result in production; a live inbox with a real history of sender-to- category routing would likely make sender a genuinely strong feature (see the recommendation to that effect above).
9. Tested day-of-week and month, derived from `date_received` , as a standalone classifier for the same reason as above: a plausible published idea (temporal features) worth checking against this data rather than assuming. It scored 0.227

accuracy, also worse than the majority-class baseline, and was not adopted. Time-of-day is not testable at all, since `date_received` in this dataset is a bare date with no timestamp. Both negative results point to the same conclusion: this dataset's metadata was not constructed to carry classification signal, so the content-based approach is correctly where the effort belongs.

Future work

1. Add a true zero-shot LLM classifier as a third comparison arm once an API key, or a locally hosted open model via Ollama, is available. This was the original plan's primary approach and would likely handle ambiguous or boundary-case emails better than similarity to centroid, particularly ones that do not closely resemble any of the 44 training examples.
2. Collect enough labeled examples in the smallest classes (9 per class minimum, 19 preferred) to make a formal class-conditional conformal prediction guarantee feasible. This would be a stronger, more regulator-defensible claim than the heuristic confidence threshold used today, and the exact numbers needed are already known (see Statistical Rigor).
3. Active learning loop. Route low-confidence predictions to a human reviewer, feed the correction back into the training set, and refit periodically. This turns the human-in-the-loop production recommendation above into an actual mechanism for the model to keep improving, instead of only ever being retrained manually.
4. Multi-label support. The current design assumes one category per email, but a real email ("please also update my beneficiary and file a claim") could legitimately belong to more than one department.